

GLM-5.1-FP8

API 接口文档

版本：v1.0

日期：2026-05-07

提供方：鸿钧计算

深圳鸿钧计算科技有限公司

目录

1. 模型概述	3
2. 接入信息	4
2.1 接口地址	4
2.2 鉴权方式	4
2.3 请求规范	4
3. 接口详细说明	5
3.1 聊天补全 — Chat Completions	5
3.1.1 基础参数	5
3.1.2 GLM-5.1-FP8 特有参数	6
3.1.3 vLLM 扩展采样参数（通过 <code>extra_body</code> 传入）	6
3.2 <code>messages</code> 消息格式	6
3.3 响应格式	7
3.3.1 非流式响应	7
3.3.2 响应字段说明	7
3.3.3 流式响应（SSE）	8
3.4 JSON 输出模式	8
3.5 工具调用（Function Calling）	9
3.5.1 <code>tools</code> 参数格式	9
3.5.2 <code>tool_choice</code> 属性	9
3.5.3 工具调用响应示例	9
3.5.4 工具结果回传	10
4. 调用示例	11
4.1 普通对话（关闭推理）	11
4.2 深度推理模式（默认开启）	11
4.3 流式输出	12
4.4 工具调用	13
4.5 结构化 JSON 输出	14
4.6 Python SDK 调用示例	15
4.7 多轮对话示例	15
5. 注意事项	17
6. 错误码说明	18
附录：模型属性列表	19

1. 模型概述

GLM-5.1-FP8 是智谱 AI (ZhipuAI) 发布的企业级大语言模型，采用稀疏混合专家 (MoE) 架构，基于 FP8 量化格式部署。总参数量 754B，每个 token 激活约 40B 参数，最大支持 128K 上下文输入能力。

核心能力:

- 深度推理与思维链 (Chain-of-Thought, reasoning_content 字段返回)
- 原生工具调用 (Tool Calling) 与并行工具调用
- 结构化 JSON 输出 (支持 JSON Schema 约束)
- 代码生成与分析
- 数学推理与复杂问题解决
- 支持中文、英文及多种语言

2. 接入信息

2.1 接口地址

类型	地址
统一入口（推荐）	https://api.ai.sztu.edu.cn/v1/chat/completions

2.2 鉴权方式

支持以下两种鉴权方式，任选其一：

方式一：Authorization Header（推荐）

Authorization: Bearer YOUR_API_KEY

2.3 请求规范

项目	说明
请求方法	POST
Content-Type	application/json
字符编码	UTF-8
超时设置	连接 60s，读写 600s（长文本请调高）

3. 接口详细说明

3.1 聊天补全 — Chat Completions

POST <https://api.ai.sztu.edu.cn/v1/chat/completions>

请求 Body 为 JSON 格式，内容如下表所示。

3.1.1 基础参数

参数名	类型	是否必填	默认值	说明
model	string	是	—	模型名称，固定传入 glm-5.1
messages	array	是	—	对话消息列表，详见 3.2 节
temperature	float	否	1.0	采样温度，范围 [0, 2]。推理场景推荐 1.0
top_p	float	否	1.0	核采样，范围 (0, 1]
max_tokens	int	否	—	最大生成 token 数
stream	bool	否	false	是否启用流式输出（SSE）
stream_options	object	否	null	流式统计配置，{"include_usage": true} 可在最后一个 chunk 返回 usage 统计
stop	string/array	否	—	停止序列，到达后停止生成，最多 4 个
n	int	否	1	每个请求生成的补全数量
seed	int	否	—	随机种子，设置后尽力确保确定性输出
frequency_penalty	float	否	0.0	频率惩罚，范围 [-2.0, 2.0]，抑制重复词频
presence_penalty	float	否	0.0	存在惩罚，范围 [-2.0, 2.0]，鼓励话题多样性
logprobs	bool	否	false	是否返回 token 级对数概率
top_logprobs	int	否	—	返回 top-k token 的 logprobs，范围 [0, 20]，需配合 logprobs=true
response_format	object	否	—	输出格式，支持 JSON 模式，详见 3.4 节
tools	array	否	—	工具/函数定义列表，详见 3.5 节
tool_choice	string/object	否	auto	工具选择策略，详见 3.5 节

3.1.2 GLM-5.1-FP8 特有参数

以下参数通过请求 Body 直接传入（OpenAI SDK 用户请通过 `extra_body` 传递）：

参数名	类型	可选值	说明
<code>chat_template_kwargs</code>	<code>object</code>	<code>{"enable_thinking": true/false}</code>	控制推理模式开关。不传时默认 <code>enable_thinking=true</code> ，即开启思维链推理； <code>false</code> =关闭推理直接输出正文

3.1.3 vLLM 扩展采样参数（通过 `extra_body` 传入）

参数名	类型	默认值	说明
<code>top_k</code>	<code>int</code>	-1（关闭）	Top-K 采样，-1 表示禁用
<code>min_p</code>	<code>float</code>	0.0	最小概率阈值采样
<code>repetition_penalty</code>	<code>float</code>	1.0	重复惩罚，>1 时抑制重复输出，推荐范围 1.0-1.2

3.2 messages 消息格式

`messages` 为对象数组，每个对象包含以下字段：

字段	类型	必填	说明
<code>role</code>	<code>string</code>	是	<code>system / user / assistant / tool</code>
<code>content</code>	<code>string/array</code>	视情况	消息内容，工具调用时 <code>assistant</code> 可为 <code>null</code>
<code>reasoning_content</code>	<code>string</code>	否	多轮对话中回传上轮思维链内容（可选，建议不回传）
<code>tool_calls</code>	<code>array</code>	否	助手消息中的工具调用列表
<code>tool_call_id</code>	<code>string</code>	否	<code>tool</code> 角色消息中指定对应的调用 ID

示例：

```
[
  { "role": "system", "content": "你是一个有帮助的 AI 助手" },
  { "role": "user", "content": "请介绍一下你自己" }
]
```

3.3 响应格式

3.3.1 非流式响应

```
{
  "id": "chatcmpl-964b85dd13387c64",
  "object": "chat.completion",
  "created": 1746604800,
  "model": "glm-5.1",
  "choices": [{
    "index": 0,
    "message": {
      "role": "assistant",
      "content": "最终回答内容",
      "reasoning_content": "思维链推理过程（GLM-5.1 特有）",
      "tool_calls": []
    },
    "finish_reason": "stop",
    "stop_reason": 154827
  }],
  "usage": {
    "prompt_tokens": 11,
    "completion_tokens": 18,
    "total_tokens": 29,
    "completion_tokens_details": { "reasoning_tokens": 0 }
  }
}
```

3.3.2 响应字段说明

字段	类型	说明
id	string	请求唯一 ID
object	string	固定为 chat.completion
created	int	Unix 时间戳
model	string	实际使用的模型名称
choices[].message.content	string	最终回答内容；工具调用时为 null
choices[].message.reasoning_content	string	思维链推理过程（enable_thinking=true 时返回，否则为 null）
choices[].message.tool_calls	array	工具调用列表，无调用时为 空数组
choices[].finish_reason	string	stop / length / tool_calls
choices[].stop_reason	int/null	触发停止的 token id，自然结束时为 154827
usage.prompt_tokens	int	输入 token 数
usage.completion_tokens	int	生成 token 数（包含 reasoning 部分）

usage.total_tokens	int	总 token 数
completion_tokens_details.reasoning_tokens	int	思维链消耗的 token 数 (reasoning 子集)

3.3.3 流式响应（SSE）

设置 stream: true 后，服务端以 Server-Sent Events 格式返回数据：

```
# 首个 chunk (role 声明)
data: {"id":"chatacmpl-xxx","object":"chat.completion.chunk","model":"glm-5.1",

"choices":[{"index":0,"delta":{"role":"assistant","content":""},"finish_reason":nul
l}]}

# 思维链阶段 (delta.reasoning_content)
data: {"id":"chatacmpl-xxx","object":"chat.completion.chunk","model":"glm-5.1",
      "choices":[{"index":0,"delta":{"reasoning_content":" 推 理 过 程 片 段
"},"finish_reason":null}]}

# 正文阶段 (delta.content)
data: {"id":"chatacmpl-xxx","object":"chat.completion.chunk","model":"glm-5.1",
      "choices":[{"index":0,"delta":{"content":" 生 成 内 容 片 段
"},"finish_reason":null}]}

# 结束 chunk
data: {"id":"chatacmpl-xxx","object":"chat.completion.chunk","model":"glm-5.1",

"choices":[{"index":0,"delta":{"content":""},"finish_reason":"stop","stop_reason":1
54827}]}

data: [DONE]
```

注意：思维链内容通过 `delta.reasoning_content` 返回，正式回答通过 `delta.content` 返回，二者顺序传输不重叠。

3.4 JSON 输出模式

通过 response_format 参数控制输出格式：

参数值	说明
{"type": "text"}	默认，纯文本输出
{"type": "json_object"}	强制模型输出合法 JSON 对象，System Prompt 中需提示返回 JSON
{"type": "json_schema", "json_schema": {...}}	按指定 JSON Schema 约束输出内容，支持 strict: true 严格模式

3.5 工具调用（Function Calling）

GLM-5.1-FP8 原生支持并行工具调用，单次可返回多个工具调用结果。

3.5.1 tools 参数格式

```
"tools": [
  {
    "type": "function",
    "function": {
      "name": "get_weather",
      "description": "获取指定城市的天气信息",
      "parameters": {
        "type": "object",
        "properties": {
          "city": { "type": "string", "description": "城市名称" }
        },
        "required": ["city"]
      }
    }
  }
]
```

3.5.2 tool_choice 属性

取值	说明
"auto"	模型自动决定是否调用工具（默认）
"none"	禁止调用任何工具
"required"	强制必须调用至少一个工具
{"type": "function", "function": {"name": "xxx"}}	强制调用指定函数

3.5.3 工具调用响应示例

```
{
  "choices": [{
    "message": {
      "role": "assistant",
      "content": null,
      "reasoning_content": null,
      "tool_calls": [{
        "id": "chatcmpl-tool-86d187278aa1521e",
        "type": "function",
        "function": {
          "name": "get_weather",
          "arguments": "{\"city\": \"深圳\"}"
        }
      }]
    },
    "finish_reason": "tool_calls",
    "stop_reason": 154829
  }]
}
```

3.5.4 工具结果回传

获取到工具调用结果后，将结果作为 **tool** 角色消息回传：

```
{
  "role": "tool",
  "tool_call_id": "chatcmpl-tool-86d187278aa1521e",
  "content": "{\"temperature\": 28, \"condition\": \"sunny\"}"
}
```

4. 调用示例

4.1 普通对话（关闭推理）

请求：

```
curl -X POST https://api.ai.sztu.edu.cn/v1/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer YOUR_API_KEY" \
-d '{
  "model": "glm-5.1",
  "messages": [{"role": "user", "content": "你好，用一句话介绍你自己"}],
  "max_tokens": 100,
  "chat_template_kwargs": {"enable_thinking": false}
}'
```

响应：

```
{
  "id": "chatcmpl-964b85dd13387c64",
  "object": "chat.completion",
  "created": 1746604800,
  "model": "glm-5.1",
  "choices": [{
    "index": 0,
    "message": {
      "role": "assistant",
      "content": "我是一个融合了海量知识、致力于为你提供高效解答与灵感的 AI 助手。",
      "reasoning_content": null,
      "tool_calls": []
    },
    "finish_reason": "stop",
    "stop_reason": 154827
  }],
  "usage": {
    "prompt_tokens": 11,
    "completion_tokens": 18,
    "total_tokens": 29,
    "completion_tokens_details": {"reasoning_tokens": 0}
  }
}
```

4.2 深度推理模式（默认开启）

请求：

```
curl -X POST https://api.ai.sztu.edu.cn/v1/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer YOUR_API_KEY" \
-d '{
  "model": "glm-5.1",
  "messages": [{"role": "user", "content": "9.11 和 9.8 哪个大? "}],
  "max_tokens": 1000
}'
```

响应:

```
{
  "id": "chatcmpl-ad8344d9643973d6",
  "object": "chat.completion",
  "created": 1746604800,
  "model": "glm-5.1",
  "choices": [{
    "index": 0,
    "message": {
      "role": "assistant",
      "content": "9.8 更大。比较小数时，先看整数部分相同，再比十分位：8 > 1，所以 9.8 > 9.11。",
      "reasoning_content": "用户在问 9.11 和 9.8 哪个大。这是一个经典的小数比较陷阱...",
      "tool_calls": []
    },
    "finish_reason": "stop",
    "stop_reason": 154827
  }],
  "usage": {
    "prompt_tokens": 15,
    "completion_tokens": 312,
    "total_tokens": 327,
    "completion_tokens_details": {"reasoning_tokens": 285}
  }
}
```

4.3 流式输出**请求:**

```
curl -N -X POST https://api.ai.sztu.edu.cn/v1/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer YOUR_API_KEY" \
-d '{
  "model": "glm-5.1",
  "messages": [{"role": "user", "content": "写一首关于深圳的短诗"}],
  "max_tokens": 200,
  "stream": true,
  "chat_template_kwargs": {"enable_thinking": false}
}'
```

响应 (SSE 片段):

```
data: {"id":"chatcmpl-9c286b50f4e0b14c","object":"chat.completion.chunk",
      "model":"glm-5.1","choices":[{"index":0,"delta":{"role":"assistant","content":""},"finish_reason":null}]}

data: {"id":"chatcmpl-9c286b50f4e0b14c","object":"chat.completion.chunk",
      "model":"glm-5.1","choices":[{"index":0,"delta":{"content":"          风"},"finish_reason":null}]}

data: {"id":"chatcmpl-9c286b50f4e0b14c","object":"chat.completion.chunk",
      "model":"glm-5.1","choices":[{"index":0,"delta":{"content":" 从 南 海 吹 来"},"finish_reason":null}]}

... (省略中间 chunks)
```

```
data: {"id":"chatcmpl-9c286b50f4e0b14c","object":"chat.completion.chunk",
      "model":"glm-5.1","choices":[{"index":0,"delta":{"content":""},"finish_reason":"stop","stop_reason":154827}]}

data: [DONE]
```

4.4 工具调用

请求:

```
curl -X POST https://api.ai.sztu.edu.cn/v1/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer YOUR_API_KEY" \
-d '{
  "model": "glm-5.1",
  "messages": [{"role": "user", "content": "今天深圳天气怎样?"}],
  "tools": [{
    "type": "function",
    "function": {
      "name": "get_weather",
      "description": "获取城市当前天气",
      "parameters": {
        "type": "object",
        "properties": {"city": {"type": "string"}},
        "required": ["city"]
      }
    }
  ]},
  "tool_choice": "auto",
  "chat_template_kwargs": {"enable_thinking": false}
}'
```

响应:

```
{
  "id": "chatcmpl-a40a1da1bbf7f834",
  "object": "chat.completion",
  "created": 1746604800,
  "model": "glm-5.1",
  "choices": [{
    "index": 0,
    "message": {
      "role": "assistant",
      "content": null,
      "reasoning_content": null,
      "tool_calls": [{
        "id": "chatcmpl-tool-86d187278aa1521e",
        "type": "function",
        "function": {
          "name": "get_weather",
          "arguments": "{\"city\": \"深圳\"}"
        }
      ]
    }
  ]},
  "finish_reason": "tool_calls",
  "stop_reason": 154829
},
  "usage": {
    "prompt_tokens": 158,
```

```
    "completion_tokens": 11,  
    "total_tokens": 169  
  }  
}
```

4.5 结构化 JSON 输出

请求:

```
curl -X POST https://api.ai.sztu.edu.cn/v1/chat/completions \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer YOUR_API_KEY" \  
-d '{  
  "model": "glm-5.1",  
  "messages": [{"role": "user", "content": "生成一个人物信息"}],  
  "response_format": {  
    "type": "json_schema",  
    "json_schema": {  
      "name": "person",  
      "strict": true,  
      "schema": {  
        "type": "object",  
        "properties": {  
          "name": {"type": "string"},  
          "age": {"type": "integer"},  
          "city": {"type": "string"}  
        },  
        "required": ["name", "age", "city"]  
      }  
    }  
  },  
  "chat_template_kwargs": {"enable_thinking": false}  
}'
```

响应:

```
{  
  "id": "chatcmpl-873ac290b9b9a184",  
  "object": "chat.completion",  
  "created": 1746604800,  
  "model": "glm-5.1",  
  "choices": [{  
    "index": 0,  
    "message": {  
      "role": "assistant",  
      "content": "{ \"name\": \"林野\", \"age\": 24, \"city\": \"深圳\" }",  
      "reasoning_content": null,  
      "tool_calls": []  
    },  
    "finish_reason": "stop",  
    "stop_reason": null  
  }],  
  "usage": {  
    "prompt_tokens": 9,  
    "completion_tokens": 27,  
    "total_tokens": 36  
  }  
}
```

4.6 Python SDK 调用示例

```
from openai import OpenAI

client = OpenAI(
    api_key="YOUR_API_KEY",
    base_url="https://api.ai.sztu.edu.cn/v1"
)

# 普通对话（关闭推理）
resp = client.chat.completions.create(
    model="glm-5.1",
    messages=[
        {"role": "system", "content": "你是一个有帮助的 AI 助手。"},
        {"role": "user", "content": "你好"},
    ],
    temperature=1.0,
    max_tokens=512,
    extra_body={"chat_template_kwargs": {"enable_thinking": False}}
)
print(resp.choices[0].message.content)

# 深度推理模式（获取思维链）
resp = client.chat.completions.create(
    model="glm-5.1",
    messages=[{"role": "user", "content": "9.11 和 9.8 哪个大? "}],
    temperature=1.0,
    max_tokens=2000
)
print('思维链: ', resp.choices[0].message.reasoning_content)
print('回答: ', resp.choices[0].message.content)
```

4.7 多轮对话示例

以下示例展示如何正确构造多轮对话的 `messages` 列表:

```
from openai import OpenAI

client = OpenAI(
    api_key="YOUR_API_KEY",
    base_url="https://api.ai.sztu.edu.cn/v1"
)

# 维护对话历史
messages = [
    {"role": "system", "content": "你是一个有帮助的 AI 助手。"}
]

# 第一轮
messages.append({"role": "user", "content": "深圳有哪些著名景点? "})
resp = client.chat.completions.create(
    model="glm-5.1",
    messages=messages,
    max_tokens=500,
    extra_body={"chat_template_kwargs": {"enable_thinking": False}}
)
assistant_reply = resp.choices[0].message.content
print('第一轮回答: ', assistant_reply)
```

```
# 将助手回复加入历史（注意：不回传 reasoning_content）
messages.append({"role": "assistant", "content": assistant_reply})

# 第二轮
messages.append({"role": "user", "content": "其中哪个最适合带小孩去? "})
resp = client.chat.completions.create(
    model="glm-5.1",
    messages=messages,
    max_tokens=500,
    extra_body={"chat_template_kwargs": {"enable_thinking": False}}
)
print('第二轮回答: ', resp.choices[0].message.content)
```


5. 注意事项

- `tool` 角色的 `content` 字段必须为字符串类型，不可传入数组或对象。
- 多轮对话中，建议不将上轮 `assistant` 消息的 `reasoning_content` 字段回传，以避免模型注意力偏移。
- 开启推理模式（`enable_thinking=true`）时，响应的 `reasoning_content` 包含完整思维链，`completion_tokens` 中包含 `reasoning_tokens` 计数。
- 流式模式下，思维链通过 `delta.reasoning_content` 返回，正文通过 `delta.content` 返回，二者顺序传输不重叠。
- `max_tokens` 同时限制思维链与正文的总生成长度，推理复杂问题时建议设置 2000 以上。
- 当前服务最大上下文长度为 131072 tokens，超出将报错。
- 工具调用时，`function.arguments` 为 JSON 字符串格式，非 `dict` 对象。
- 不传 `chat_template_kwargs` 时，模型默认开启思维链（`enable_thinking=true`），响应会包含 `reasoning_content` 字段。

6. 错误码说明

HTTP 状态码	错误类型	说明及处理建议
400	Bad Request	请求参数格式错误，检查 JSON 格式及必填字段
401	Unauthorized	API Key 无效或未传入，检查 Authorization 头部
422	Unprocessable Entity	参数值超出范围或 model 字段不匹配，检查参数取值
429	Too Many Requests	请求频率超限，建议添加重试逻辑（指数退避）
500	Internal Server Error	服务端内部错误，可重试，持续出现请联系技术支持
503	Service Unavailable	服务暂时不可用，通常为短暂过载，稍后重试

错误响应体格式：

```
{
  "object": "error",
  "message": "Model not found: glm-5.2",
  "type": "invalid_request_error",
  "code": 400
}
```

附录：模型属性列表

属性	值
模型名称	glm-5.1
总参数量	754B
每 token 激活参数	~40B
最大上下文	131072 tokens（128K）
最大输出	131072 tokens
默认 temperature	1.0
默认 top_p	0.95
支持语言	中文、英文及多种语言
支持工具调用	是（支持并行调用）
支持思维链	是（reasoning_content 字段，默认开启）
支持 JSON Mode	是
支持流式输出	是（SSE）
支持图片输入	否
量化格式	FP8
架构	稀疏混合专家（MoE）